Université de Versailles Saint-Quentin-en-Yvelines

Master 1 CHPS – CHP203 Techniques d'optimisation parallèle 2025/2026

Rapport de projet : optimisation hybride MPI+OpenMP d'un solveur LBM D2Q9

Rue de vortex de Kármán

Auteurs :

Xiang BIAN Hao QIAN

Enseignants :

Gabriel Dos Santos Hugo Taboada

Dépôt Git versionné : https://github.com/xiang024/top_projet

27 avril 2026

Table des matières

Résumé **1**

1 Introduction et contexte LBM D2Q9 **1**

2 Méthodologie expérimentale **1**
2.1 Convention de référence 1

3 Configuration matérielle et logicielle **2**

4 Validation de la correction **2**

5 Optimisations MPI **3**
5.1 Correction parallèle 3
5.2 Suppression des synchronisations inutiles 3
5.3 Passage progressif à MPI_Sendrecv 3
5.4 Profil après communication 4

6 Optimisations noyaux de calcul **4**
6.1 Collision D2Q9 spécialisée 4
6.2 Conditions aux limites 4
6.3 Propagation D2Q9 spécialisée 4
6.4 Cache des cellules spéciales 5

7 Analyse OpenMP et hybride MPI+OpenMP **5**
7.1 OpenMP seul 5
7.2 Campagne hybride finale 5

8 Résultats secondaires sur AMD Ryzen 7 5800H **6**

9 Discussion critique **7**

10 Conclusion **7**

A Commandes de reproduction **7**
A.1 Compilation 7
A.2 Exécutions de référence 7
A.3 Validation numérique 7
A.4 Analyse OpenMP 7
A.5 Campagne finale 8

B Correspondance avec les consignes **8**

Résumé

Ce rapport présente une démarche d'optimisation incrémentale d'un solveur LBM D2Q9 hybride MPI+OpenMP simulant une rue de vortex de Kármán. À chaque étape, un profil par phase ou une mesure globale est utilisé pour identifier le goulot dominant, puis la modification est validée par checksum avant d'être mesurée. Les principales améliorations proviennent de la correction des échanges MPI, de l'utilisation de `MPI_Sendrecv` et de `MPI_Datatype`, puis de la spécialisation des noyaux collision et propagation. En prenant comme référence prudente la version corrigée mesurée en `np=1` après suppression des synchronisations inutiles, le meilleur lancement final réduit le temps réel de 195,98 s à 4,83 s, soit environ 40.6×. La campagne finale montre que, pour le cas étudié et sur la machine principale, la configuration MPI pure `OMP_NUM_THREADS=1 mpirun -np 16` est la plus performante, avec 9889,97 MLUPS affichés. Les configurations hybrides sont moins efficaces en raison d'une saturation rapide de l'OpenMP.

1 Introduction et contexte LBM D2Q9

Le code étudié simule l'écoulement d'un fluide dans un canal contenant un obstacle circulaire, configuration qui produit une rue de vortex de Kármán. La méthode utilisée est la méthode de Boltzmann sur réseau, ou LBM. Dans le schéma D2Q9, chaque cellule de la grille porte neuf populations microscopiques f_i, associées aux directions discrètes $\mathbf{e}_i$.

Les grandeurs macroscopiques sont reconstruites par

$$\rho(\mathbf{x},t) = \sum_{i=0}^{8} f_i(\mathbf{x},t), \qquad \mathbf{u}(\mathbf{x},t) = \frac{1}{\rho}\sum_{i=0}^{8} f_i(\mathbf{x},t)\mathbf{e}_i. \tag{1}$$

L'étape de collision relaxe les populations vers un équilibre f_i^{eq} :

$$f_i^{new} = f_i - \frac{1}{\tau}\left(f_i - f_i^{eq}\right). \tag{2}$$

À chaque itération, le solveur applique les conditions aux limites et à l'obstacle, calcule les collisions, échange les halos entre sous-domaines MPI, puis propage les populations vers les cellules voisines.

2 Méthodologie expérimentale

Le protocole a été conservé identique entre les étapes afin de rendre les mesures comparables. Les temps globaux sont mesurés avec `/usr/bin/time` ; les tests de scalabilité utilisent principalement `mpirun -np 1` et `mpirun -np 4` pendant l'optimisation, puis plusieurs couples MPI/OpenMP pour la campagne finale.

La configuration de simulation utilisée est :

Paramètre	Valeur
Itérations	20000
Maillage	800×160
Obstacle	$(x, y, r) = (100.0, 80.0, 12.0)$
Reynolds	96
Vitesse maximale d'entrée	0.16
Sortie	`results.raw`, toutes les 50 itérations

La démarche imposée par le projet a été suivie à chaque étape : outil, diagnostic, optimisation, mesure, puis validation. Les profils par phase ont été produits avec `MPI_Wtime` et agrégés par `MPI_Reduce(..., MPI_MAX)` afin de retenir le temps du rang limitant.

2.1 Convention de référence

Plusieurs références apparaissent dans le rapport, car elles ne servent pas au même usage. Pour les tableaux de communication, le speedup S_4 est calculé localement à partir de la même version du code en `np=1` et `np=4` ; il mesure donc la scalabilité MPI de cette étape, pas le gain cumulé du projet. Pour le gain global final, la référence retenue est la première version parallèle corrigée et validée, mesurée à 195,98 s en `np=1`. Pour l'étude OpenMP et la campagne finale, les speedups sont calculés à partir de la version déjà optimisée en `mpirun -np 1`, `OMP=1`, soit 45,76 s ; ils comparent donc uniquement les choix de lancement MPI/OpenMP.

Table 1 – Synthèse cumulative des principales étapes d'optimisation.

Version	Modification principale	`np=1` (s)	`np=4`/final (s)	Impact principal	Validation
Correction + barrières	Parallèle corrigé et synchronisations inutiles supprimées	195.98	104.88	Référence prudente validée	ok
V4	Type MPI `row_type`	193.07	39.72	Fort gain sur les halos verticaux	ok
V5	Échanges redondants supprimés	188.36	40.43	Communication simplifiée	ok
Collision rapide	Noyau D2Q9 spécialisé	79.56	17.87	Goulot collision réduit	ok
Propagation spécialisée	Déplacements D2Q9 explicites	46.00	12.62	Second goulot réduit	ok
Cache cellules spéciales	Liste des cellules spéciales réutilisée	45.82	12.32	Scan global évité	ok
Campagne finale	`OMP=1 mpirun -np 16`	–	4.832	9889,97 MLUPS affichés	ok

Ce tableau sert de fil conducteur : les sections suivantes détaillent pour chaque étape l'outil utilisé, le diagnostic, la modification apportée, l'impact mesuré et la validation.

3 Configuration matérielle et logicielle

La machine principale utilisée pour les résultats finaux est décrite dans le tableau 2. Les résultats sur AMD Ryzen 7 5800H sont conservés uniquement comme test secondaire de portabilité.

Table 2 – Configuration de la machine principale de benchmark.

Élément	Valeur
CPU	AMD Ryzen 9 8945HX with Radeon Graphics
Architecture	x86_64, 1 socket, 16 coeurs physiques, 32 threads, SMT=2
Fréquence	400 MHz – 5461 MHz
Caches	L1d 512 KiB × 16, L1i 512 KiB × 16, L2 16 MiB, L3 64 MiB
SIMD	SSE/SSE2/SSE4.1/SSE4.2, AVX, AVX2, AVX-512
Mémoire	31 GiB ; fréquence DRAM non mesurée
Interconnexion	mono-noeud, sans réseau inter-noeud
OS	Ubuntu 22.04.5 LTS, noyau Linux 6.8.0-110-generic
Page size	4096 octets
Compilateur	g++ 11.4.0
CMake	3.31.11
MPI / OpenMP	Open MPI 4.1.2, OpenMP via `find_package(OpenMP)`
Mode de compilation	Release
Flags principaux	`-O3 -DNDEBUG -ffp-contract=off -march=native -fno-math-errno -DLBM_FAST_COLLISION=1`

4 Validation de la correction

La contrainte principale du projet est de conserver exactement les résultats de la baseline. La première validation a montré une différence importante entre les exécutions : le cas `mpirun -np 1` passait le checksum, alors que le cas `mpirun -np 4` ne le passait pas. Le problème n'était donc pas seulement une question de performance, mais d'abord une question de validité parallèle. Cette observation a motivé l'étape de correction décrite en section 5.1.

Les corrections de validité parallèle ont porté sur trois points principaux :

— l'utilisation des coordonnées globales pour les conditions d'entrée ;

— l'utilisation de la hauteur globale du domaine dans le profil de Poiseuille ;

— la correction des échanges de halos verticaux, afin de recevoir le bon nombre de données et de couvrir toute la zone intérieure concernée.

Après ces corrections, le checksum `np=4` est redevenu valide.

Après cette correction, puis après chaque optimisation conservée, la sortie `results.raw` a été comparée à `ref_results.raw` :

```
PYTHONPATH=.pydeps:src python3 -m lbm_viz \
  --check ref_results.raw results.raw \
```

```
--display-bin-path build/top.display
```

Les journaux indiquent un checksum `ok` après la correction parallèle, puis pour les étapes de communication `Sendrecv`, de `row_type`, de collision rapide, de propagation spécialisée et de cache des cellules spéciales. La validation numérique a donc été intégrée au protocole expérimental : aucune mesure de performance n'a été considérée comme exploitable sans checksum correct.

5 Optimisations MPI

5.1 Correction parallèle

Avant d'interpréter les performances MPI, la validité numérique en parallèle a été restaurée. Le diagnostic reposait sur `lbm_viz -check` : la version mono-processus passait, mais la version `np=4` dérivait. Une étape de correction de la parallélisation a donc été réalisée sur l'initialisation, les conditions aux limites et les échanges de halos. Après cette correction, le checksum `np=4` est redevenu valide. Les optimisations de performance décrites ensuite partent de cette version corrigée.

5.2 Suppression des synchronisations inutiles

Le code initial contenait plusieurs synchronisations globales dans la boucle temporelle et dans la communication. Sur cette version très désoptimisée, une exécution complète pouvait durer environ cinq heures, ce qui rendait l'expérimentation difficile. L'outil principal est ici `/usr/bin/time`, complété par la revue des appels MPI. Le diagnostic est que ces barrières imposent des attentes globales sans dépendance numérique utile. L'optimisation supprime trois `MPI_Barrier` dans la boucle principale et un `MPI_Syncall` inutile en fin d'échange de halo. Après ce nettoyage, deux tests `np=1` ont donné des temps proches, 3m11.144s et 3m08.480s. Ces deux valeurs sont donc interprétées comme deux mesures de contrôle du même état optimisé, et non comme un gain de 3m11 vers 3m08. L'intérêt de cette étape est surtout de supprimer les attentes extrêmes et de rendre les mesures suivantes praticables. La validation par `lbm_viz -check` reste `ok`.

5.3 Passage progressif à MPI_Sendrecv

Les échanges de halos ont ensuite été réécrits progressivement avec `MPI_Sendrecv`. Les mesures `/usr/bin/time` sur `np=1` et `np=4`, puis les profils `MPI_Wtime` agrégés par `MPI_Reduce(..., MPI_MAX)`, ont montré que la communication restait coûteuse. Le diagnostic porte sur des appariements `Send`/`Recv` fragiles et des transferts redondants. L'optimisation introduit `MPI_Sendrecv`, puis un `MPI_Datatype` `row_type`, avant de supprimer des échanges inutiles. L'impact est mesuré dans le tableau 3, et chaque étape conservée garde un checksum `ok`.

Table 3 – Étapes de communication MPI. Les speedups et efficacités sont calculés à partir des temps `np=1`/`np=4` de chaque version.

Étape	`np=1` (s)	`np=4` (s)	S_4	E_4	Validation
Après correction parallèle et désynchronisation conjointe (`MPI_Barrier` + `MPI_Syncall`)	195.98	104.88	1.87	46.7%	ok
V1, halos haut/bas en `Sendrecv`	193.46	69.11	2.80	70.0%	ok
V2, halos gauche/droite en `Sendrecv`	202.57	65.64	3.09	77.1%	ok
V3, diagonales en `Sendrecv`	187.49	68.31	2.74	68.5%	ok
V4, type MPI `row_type`	193.07	39.72	4.86	121.5%	ok
V5, échanges redondants supprimés	188.36	40.43	4.66	116.5%	ok

Convention de lecture : dans ce tableau, S_4 et E_4 sont calculés localement avec les temps `np=1`/`np=4` de chaque version. Le gain global final, lui, est calculé séparément entre la référence validée à 195,98 s et le meilleur temps final de 4,83 s.

La tentative V3 est instructive : elle améliore `np=1`, mais dégrade `np=4` de 65.64s à 68.31s. Elle a donc été analysée comme une régression parallèle, non comme un succès. La règle de décision utilisée est la suivante : une modification est conservée comme optimisation si le checksum reste `ok` et si `np=4` s'améliore ; une régression supérieure à environ 2% est marquée comme candidate au retour arrière, sauf si une étape suivante exploite cette restructuration et rétablit un gain net. V3 relève de ce second cas et n'est pas revendiquée seule ; V5, avec une régression d'environ 1.8%, est conservée surtout pour simplifier le chemin de communication. La forte amélioration de V4 vient du passage des halos verticaux à un échange de ligne complète via `MPI_Datatype`. La valeur $E_4 > 100\%$ est interprétée prudemment : elle peut venir d'effets de cache et du changement de granularité, et mériterait des répétitions statistiques.

5.4 Profil après communication

Une fois la communication fortement réduite, le profil `np=4` montre que le goulot se déplace vers le calcul :

Table 4 – Profil par phase après les principales optimisations de communication, `np=4`.

Phase	Temps (s)	Part
`special_cells`	0.357	0.9%
`collision`	29.784	71.9%
`halo_exchange`	3.887	9.4%
`propagation`	6.361	15.4%
`save_frame`	1.037	2.5%

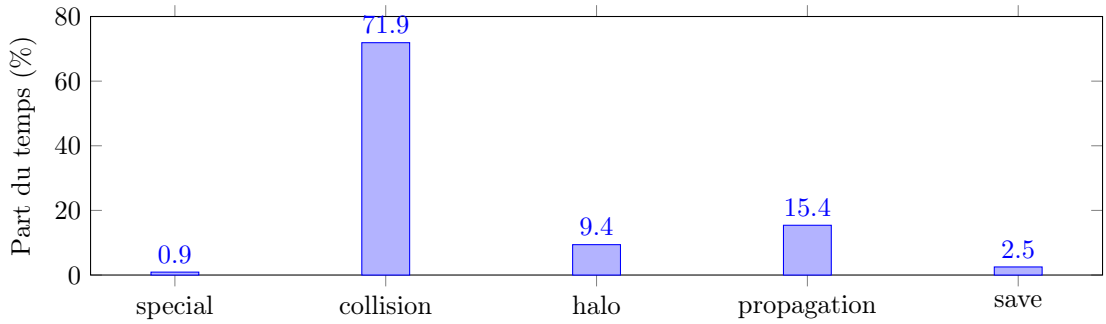

Figure 1 – Le profil après communication identifie `collision` comme nouveau goulot principal.

6 Optimisations noyaux de calcul

6.1 Collision D2Q9 spécialisée

Le profil précédent, obtenu avec `MPI_Wtime` et une agrégation au maximum des rangs, montre que le noyau de collision domine le temps d'exécution. Le diagnostic est donc un goulot de calcul arithmétique dans le noyau D2Q9. L'optimisation a consisté à introduire une voie rapide D2Q9 activée par `LBM_FAST_COLLISION`, avec calculs déroulés explicitement et conservation d'un chemin de référence pour comparaison.

Table 5 – Impact de l'optimisation de collision.

Cas	Avant (s)	Après (s)	Gain
`np=1`	188.36	79.56	2.37×
`np=4`	40.43	17.87	2.26×

L'impact est un gain de 2.37× en `np=1` et 2.26× en `np=4`. Le checksum reste `ok`. Après cette étape, la part de `collision` baisse de 71.9% à 54.7% en `np=4`, tandis que `propagation` remonte mécaniquement en part relative, de 15.4% à 28.4%.

6.2 Conditions aux limites

Une étape d'alignement des formules `bounce_back`, `inflow` et `outflow` a été testée. Elle est correcte numériquement mais n'apporte pas de gain mesurable : `np=1` passe de 79.56s à 80.13s, et `np=4` de 17.87s à 17.96s. Cette tentative est conservée pour la cohérence du code, mais elle n'est pas revendiquée comme optimisation de performance.

6.3 Propagation D2Q9 spécialisée

Après la collision rapide, le profil par phase identifie la propagation comme second goulot. Le diagnostic est que le noyau générique conserve trois boucles et des tests de bord dans un schéma pourtant fixé à D2Q9. L'optimisation remplace ce noyau par une boucle spécialisée, utilisant des indices continus et des déplacements explicites des neuf directions.

Les mesures de cette étape correspondent à la version intermédiaire utilisée pour isoler le coût de propagation ; elles ne doivent donc pas être lues comme la continuation directe du tableau précédent, mais comme une comparaison avant/après sur le même point de mesure.

Table 6 – Impact de l'optimisation de propagation.

Cas	Avant (s)	Après (s)	Gain
`np=1`	62.90	46.00	1.37×
`np=4`	16.41	12.62	1.30×

L'impact mesuré est de 1.37× en `np=1` et 1.30× en `np=4`. Le checksum reste `ok`. La part de `propagation` chute de 43.8% à 24.7% en `np=1`, et de 29.4% à 12.0% en `np=4`.

6.4 Cache des cellules spéciales

Le profil et la revue de code ont ensuite montré que `special_cells` rescannait tout le domaine à chaque pas de temps alors que seules les cellules de bord ou d'obstacle nécessitent un traitement particulier. Le diagnostic est donc un coût répété sur une géométrie fixe. Un cache `special_x/special_y/special_type` a été construit au premier appel, puis réutilisé.

Table 7 – Impact du cache des cellules spéciales.

Cas	Avant (s)	Après (s)	Gain global
`np=1`	46.00	45.82	0.4%
`np=4`	12.62	12.32	2.4%

Le gain global est modéré, mais la phase ciblée diminue fortement : en `np=4`, `special_cells` passe de 0.329s, soit 2.6%, à 0.037s, soit 0.3%. La sortie est de nouveau contrôlée par `lbm_viz -check`, avec checksum `ok`.

7 Analyse OpenMP et hybride MPI+OpenMP

7.1 OpenMP seul

Pour isoler OpenMP, les tests ont été réalisés avec `mpirun -np 1`, `OMP_PROC_BIND=close` et `OMP_PLACES=cores`. Le meilleur compromis est `OMP_NUM_THREADS=2`.

Table 8 – Scalabilité OpenMP seule, avec `mpirun -np 1`.

Threads OpenMP	Temps (s)	Speedup	Efficacité
1	45.76	1.00	100.0%
2	33.88	1.35	67.5%
4	34.88	1.31	32.8%
8	35.06	1.30	16.3%

La scalabilité limitée d'OpenMP suggère que les noyaux optimisés ne sont plus uniquement limités par le calcul arithmétique. Chaque mise à jour de cellule manipule plusieurs populations D2Q9, ce qui génère un trafic mémoire important. L'augmentation du nombre de threads accroît la pression sur la bande passante mémoire et ajoute un coût de synchronisation/scheduling. Cela explique que, sur cette machine et pour ce cas, plusieurs processus MPI avec un seul thread chacun soient plus efficaces que peu de processus avec beaucoup de threads. Le profil confirme que `collision` reste majoritaire : 71.6% avec un thread, puis environ 73% avec plusieurs threads.

7.2 Campagne hybride finale

La campagne finale combine MPI et OpenMP. La FOM est l'indicateur affiché par le programme et peut ne pas recouvrir exactement le même périmètre que le temps réel mesuré par `/usr/bin/time`. Le classement final est donc principalement interprété à partir du temps réel, tandis que la FOM sert d'indicateur complémentaire de performance du noyau. Les speedups ci-dessous sont calculés à partir du temps OpenMP seul à un thread, soit 45.76s.

Table 9 – Résultats finaux sur la machine principale.

Lancement	FOM (MLUPS)	Temps réel (s)	Speedup	Efficacité
`OMP=8 mpirun -np 2`	303.42	17.564	2.61	16.3%
`OMP=4 mpirun -np 4`	2327.50	5.084	9.00	56.3%
`OMP=2 mpirun -np 8`	4286.98	5.431	8.43	52.7%
`OMP=1 mpirun -np 16`	9889.97	4.832	9.47	59.2%
`OMP=1 mpirun -np 8`	3144.63	7.143	6.41	80.1%
`OMP=2 mpirun -np 4`	1618.57	6.958	6.58	82.2%
`OMP=4 mpirun -np 2`	308.84	17.194	2.66	33.3%

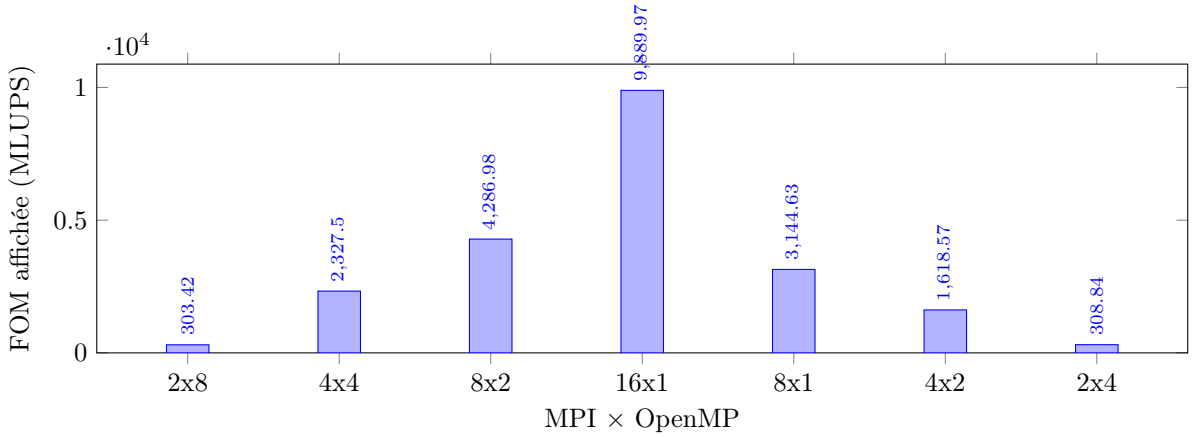

Figure 2 – FOM finale sur la machine principale. Le meilleur lancement mesuré est `16x1`.

Le meilleur résultat principal est donc `OMP_NUM_THREADS=1 mpirun -np 16`. Les configurations hybrides avec beaucoup de threads par processus sont moins efficaces, ce qui est cohérent avec la saturation OpenMP observée séparément. Le checksum final reste `ok`.

8 Résultats secondaires sur AMD Ryzen 7 5800H

Les essais sur AMD Ryzen 7 5800H ne sont pas utilisés pour choisir le meilleur lancement principal. Ils servent uniquement à vérifier que les tendances restent raisonnables sur une autre machine.

Table 10 – Configuration de la machine secondaire AMD Ryzen 7 5800H utilisée pour les tests de portabilité.

Élément	Valeur
CPU	AMD Ryzen 7 5800H with Radeon Graphics
Architecture	x86_64, 1 socket, 8 coeurs physiques, 16 threads, SMT=2
Fréquence	403 MHz – 4465 MHz
Caches	L1d 256 KiB × 8, L1i 256 KiB × 8, L2 4 MiB, L3 16 MiB
SIMD	SSE/SSE2/SSE4.1/SSE4.2, AVX, AVX2
Mémoire	15 GiB ; fréquence DRAM non mesurée
Interconnexion	mono-noeud, sans réseau inter-noeud
OS	Ubuntu 24.04, noyau Linux 6.17.0-22-generic
Compilateur	g++ 13.3.0
CMake	3.28.3
MPI / OpenMP	Open MPI 4.1.6, OpenMP GNU

Table 11 – Tests secondaires de portabilité sur AMD Ryzen 7 5800H.

Lancement	FOM (MLUPS)	Temps réel (s)
`OMP=8 mpirun -np 2`	145.41	37.118
`OMP=4 mpirun -np 4`	510.77	21.954
`OMP=2 mpirun -np 8`	941.36	23.580
`OMP=2 mpirun -np 4`	581.09	19.515

9 Discussion critique

Les optimisations les plus rentables ont été celles qui attaquaient un goulot mesuré. La première phase a supprimé conjointement des synchronisations globales inutiles, puis les versions suivantes ont réduit le coût des halos. Une fois la communication descendue à 9.4%, continuer à l'optimiser aurait eu un rendement plus faible ; le profil a correctement déplacé l'effort vers `collision`, puis `propagation`. Cette observation est cohérente avec la loi d'Amdahl : une fois qu'une phase représente une faible fraction du temps total, son optimisation supplémentaire a un effet limité sur le temps global.

Plusieurs résultats appellent une lecture prudente. D'abord, V3 est une tentative correcte mais défavorable à `np=4` ; elle montre que rendre le code plus uniforme ne garantit pas un gain. Ensuite, l'efficacité supérieure à 100% observée après `row_type` n'est pas une preuve de scalabilité parfaite : elle peut s'expliquer par le cache et par la réduction brutale du nombre d'appels MPI. Enfin, l'hybride MPI+OpenMP n'est pas automatiquement meilleur que MPI pur. Sur la machine principale, `OMP=1, np=16` domine, tandis que les configurations à 8 ou 4 threads par processus sont pénalisées.

Les limites de l'étude sont les suivantes : certaines mesures ne sont disponibles qu'en exécution unique, les statistiques de variance ne sont donc pas disponibles, et la fréquence DRAM n'a pas été mesurée. Une étude plus robuste répéterait chaque point au moins trois fois, en isolant les phases d'écriture et en fixant davantage la fréquence CPU.

10 Conclusion

Le projet a transformé une version initialement désoptimisée en une version validée et nettement plus performante. La stratégie a été itérative : corriger d'abord la validité parallèle, supprimer conjointement les synchronisations globales inutiles de la boucle principale et de la communication, réduire les halos par `MPI_Sendrecv` et `MPI_Datatype`, puis optimiser les noyaux dominants après profiling. Le saut initial de performance provient de cette suppression conjointe, et non d'un unique appel MPI. En prenant comme référence prudente la version corrigée mesurée en `np=1` après suppression des synchronisations inutiles, le meilleur lancement final réduit le temps réel de 195,98 s à 4,83 s, soit un gain global d'environ 40.6×. Le meilleur lancement mesuré sur la machine principale est `OMP_NUM_THREADS=1 mpirun -np 16`, avec 9889,97 MLUPS affichés et 4,83 s de temps réel. La conclusion technique est que, pour ce cas et cette machine, les optimisations MPI et noyaux calcul dominent, tandis que l'OpenMP sature rapidement.

A Commandes de reproduction

A.1 Compilation

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release \
  -DCMAKE_CXX_FLAGS="-DLBM_FAST_COLLISION=1"
cmake --build build -j
```

A.2 Exécutions de référence

```
/usr/bin/time -f 'np=1 | %E real, %U user, %S sys' \
  mpirun -np 1 ./build/top.lbm-exe config.txt

/usr/bin/time -f 'np=4 | %E real, %U user, %S sys' \
  mpirun -np 4 ./build/top.lbm-exe config.txt
```

A.3 Validation numérique

```
PYTHONPATH=.pydeps:src python3 -m lbm_viz \
  --check ref_results.raw results.raw \
  --display-bin-path build/top.display
```

A.4 Analyse OpenMP

```
export OMP_PROC_BIND=close
export OMP_PLACES=cores

export OMP_NUM_THREADS=1
/usr/bin/time -f 'omp=1 | %E real, %U user, %S sys' \
  mpirun -np 1 ./build/top.lbm-exe config.txt

export OMP_NUM_THREADS=2
/usr/bin/time -f 'omp=2 | %E real, %U user, %S sys' \
  mpirun -np 1 ./build/top.lbm-exe config.txt

export OMP_NUM_THREADS=4
/usr/bin/time -f 'omp=4 | %E real, %U user, %S sys' \
  mpirun -np 1 ./build/top.lbm-exe config.txt

export OMP_NUM_THREADS=8
/usr/bin/time -f 'omp=8 | %E real, %U user, %S sys' \
  mpirun -np 1 ./build/top.lbm-exe config.txt
```

A.5 Campagne finale

```
OMP_NUM_THREADS=8 mpirun -np 2 ./build/top.lbm-exe config.txt
OMP_NUM_THREADS=4 mpirun -np 4 ./build/top.lbm-exe config.txt
OMP_NUM_THREADS=2 mpirun -np 8 ./build/top.lbm-exe config.txt
OMP_NUM_THREADS=1 mpirun -np 16 ./build/top.lbm-exe config.txt
OMP_NUM_THREADS=1 mpirun -np 8 ./build/top.lbm-exe config.txt
OMP_NUM_THREADS=2 mpirun -np 4 ./build/top.lbm-exe config.txt
OMP_NUM_THREADS=4 mpirun -np 2 ./build/top.lbm-exe config.txt
```

B Correspondance avec les consignes

- Rapport en français, structuré pour une limite de 10 pages hors garde, table et annexes.
- Lien Git inclus sous forme `\url{...}` : https://github.com/xiang024/top_projet.
- Démarche outil, diagnostic, optimisation, mesure et validation.
- Tentatives non pleinement réussies documentées : V3, alignement des conditions aux limites, saturation OpenMP.
- Limite explicitée : certaines mesures sont en exécution unique ; les conclusions associées sont donc interprétées avec prudence.
- Configuration matérielle et logicielle incluse.
- Validation par checksum incluse.